

Manual de Usuario

KGeoMIP / KQNodes

Sistema de Análisis de Irreducibilidad Sistémica
mediante K-Particiones

Versión 2.0

Fecha: 2026-06-13

Proyecto K-QGMIP

Universidad de Caldas — Facultad de Ingeniería

Análisis y Diseño de Algoritmos — 2026-1

Índice

1. Introducción y visión general	3
1.1. Qué hace el software	3
1.2. Para qué sirve	3
1.3. Conceptos básicos	4
1.4. Capacidades y limitaciones	4
2. Requisitos del sistema	6
2.1. Sistema operativo	6
2.2. Hardware	6
2.3. Software	7
3. Instalación paso a paso	8
3.1. Descarga del proyecto	8
3.2. Instalación de Python	8
3.3. Instalación del gestor de entornos uv	9
3.4. Creación del entorno virtual e instalación de dependencias	9
3.5. Verificación de instalación correcta	9
3.6. Solución de problemas comunes de instalación	10
4. Video tutorial	11
4.1. Enlace al video	11
5. Guía de uso básico	12
5.1. Preparación de datos de entrada	12
5.1.1. Archivos de Matriz de Probabilidad de Transición (TPM)	12
5.1.2. Archivo de configuración Excel	13
5.1.3. Estructura de directorios	13
5.2. Ejecución básica	13
5.2.1. Ejecución desde Excel (GeometricSIA + KPartition)	13
5.2.2. Ejecución de un caso específico (debug)	14
5.2.3. Ejecución del benchmark completo	14
5.3. Interpretación de resultados	15
5.3.1. Salida en consola	15
5.3.2. Archivos de resultados Excel	15
5.3.3. Archivos de log	16
5.4. Ejecución de la suite de pruebas	16
6. Opciones y parámetros avanzados	17
6.1. Parámetros del benchmark	17
6.2. Estrategias disponibles	17
6.3. Heurísticas de KPartitionSIA	18
6.4. Parámetro de timeout	18
6.5. Optimización de rendimiento	19
6.6. Cache de resultados	20

7. Solución de problemas	21
7.1. Errores comunes	21
7.2. Problemas de instalación	21
7.3. Problemas de ejecución	22
7.4. Problemas con datos de entrada	22
7.5. Preguntas frecuentes (FAQ)	22
8. Ejemplos y tutoriales	24
8.1. Tutorial básico: Sistema de 3 nodos (N3C)	24
8.2. Caso de estudio intermedio: Sistema de 10 nodos (N10A)	24
8.3. Ejemplo avanzado: Benchmark con $n = 20$	25
8.4. Ejemplo $n = 25$: Benchmark aproximado con KL+MC-EMD	26
9. Referencia rápida	27
9.1. Comandos principales	27
9.2. Tabla de parámetros	27
9.3. Formato de archivos	27
9.4. Glosario	28

1. Introducción y visión general

1.1. Qué hace el software

KGeoMIP (K Geometric Minimum Information Partition) es una herramienta de software que analiza sistemas complejos representados como redes de nodos interconectados. Su función principal es encontrar la mejor manera de dividir un sistema en partes (k partes) de forma que se pierda la menor cantidad de información posible al separarlas.

Imagine que tiene un motor de automóvil y quiere entender cómo funcionan sus piezas en conjunto. Si separa el motor en partes, cada parte ya no tiene acceso a toda la información del sistema completo. KGeoMIP encuentra la forma de separar el sistema que minimiza esta pérdida de información, revelando cuáles son las partes más integradas del sistema.

El software incluye varias estrategias de búsqueda:

- Para división en 2 partes (bipartición): algoritmos rápido y preciso.
- Para división en 3, 4 o 5 partes (k -partición): heurísticas inteligentes que encuentran buenas soluciones sin tener que probar todas las posibilidades (que serían millones).

1.2. Para qué sirve

KGeoMIP tiene aplicaciones en diversos campos:

Neurociencia teórica: Analizar redes neuronales para entender cómo se integra la información en el cerebro, conceptos relacionados con la teoría de la conciencia.

Biología de sistemas: Identificar módulos funcionales en redes metabólicas o de regulación genética.

Inteligencia artificial: Evaluar el grado de integración de información en redes neuronales artificiales.

Diseño de sistemas: Determinar la estructura óptima de sistemas complejos donde se busca 平衡 (equilibrar) la integración y la modularidad.

Casos de uso típicos:

- Un investigador quiere saber si un sistema de 10 neuronas artificiales está más integrado que otro, comparando sus valores de pérdida.
- Un estudiante necesita determinar la mejor partición en 3 partes de una red de 15 nodos para un trabajo de investigación.
- Un ingeniero evalúa cómo cambia la integración de un sistema al modificar sus conexiones internas.

1.3. Conceptos básicos

Sistema: Conjunto de nodos (elementos) que interactúan entre sí. Cada nodo puede estar en estado 0 o 1 en cada instante de tiempo.

TPM (Matriz de Probabilidad de Transición): Una tabla que describe cómo cambia el sistema de un estado a otro. Por ejemplo, para un sistema de 3 nodos, la TPM indica la probabilidad de cada posible estado futuro dado el estado actual.

Particion: Una forma de dividir el conjunto de nodos en grupos separados. Por ejemplo, si tiene 4 nodos $\{A, B, C, D\}$, una partición en 2 partes podría ser $\{A, B\}$ y $\{C, D\}$. Una partición en 3 partes podría ser $\{A\}$, $\{B, C\}$, $\{D\}$.

Pérdida de información (ϕ): Un número que mide cuánta información se pierde al separar el sistema en partes. Un valor de 0 significa que no se pierde información (las partes son independientes). Valores más altos indican mayor pérdida.

MIP (Partición de Mínima Información): La mejor partición posible, aquella que produce la menor pérdida de información.

k : El número de partes en que se divide el sistema. KGeoMIP trabaja con $k = 2, 3, 4$ o 5 .

EMD (Distancia del Móvil de Tierra): La fórmula matemática que usa el software para medir la diferencia entre el sistema original y el sistema particionado. Se puede entender como “el esfuerzo mínimo necesario para transformar una distribución de probabilidad en otra”.

1.4. Capacidades y limitaciones

Capacidades:

- Analizar sistemas de hasta 15 nodos con resultados precisos en segundos o minutos.
- Analizar sistemas de 20 nodos con tiempos de cómputo de horas (resultados viables con heurísticas especiales).
- Analizar sistemas de 25 nodos mediante el algoritmo de aproximación KL+MC-EMD, que omite el paso exacto y emplea partición aleatoria con refinamiento Kernighan-Lin y estimación Monte Carlo de la EMD.
- Encontrar la mejor partición para $k = 2$ de manera exacta (usando GeometricSIA) o aproximada (QNodes).
- Encontrar buenas particiones para $k = 3, 4, 5$ usando heurísticas inteligentes (Greedy, Kernighan-Lin, MCTS+MC-EMD, KL+MC-EMD).
- Procesar múltiples casos automáticamente desde un archivo Excel.
- Generar reportes detallados en formato Excel con resultados, tiempos y métricas de rendimiento.

Limitaciones:

- No puede analizar sistemas de más de 25 nodos en hardware convencional (el tiempo de cómputo crece exponencialmente).
- Las heurísticas para $k \geq 3$ no garantizan encontrar la partición absolutamente mejor, sino una muy buena (óptimo local).
- Para $n = 25$, los resultados son inherentemente aproximados (KL+MC-EMD); no existe referencia exacta para validación cruzada.
- Para $k = 2$, el algoritmo GeometricSIA es exacto, pero QNodes es heurístico (aproximado).
- No incluye interfaz gráfica de usuario (GUI); opera por línea de comandos.
- Requiere conocimientos básicos de Python y línea de comandos para su instalación y uso.

2. Requisitos del sistema

2.1. Sistema operativo

KGeoMIP ha sido probado y funciona correctamente en:

- Windows 10 y 11 (validado extensivamente)
- Linux (Ubuntu 20.04+, Debian 11+, Fedora 36+)
- macOS (no probado oficialmente, pero debería funcionar con Python y las dependencias estándar)

2.2. Hardware

Requisitos mínimos:

- Procesador: CPU de 2 núcleos, 2.0 GHz
- Memoria RAM: 8 GB (suficiente para sistemas de hasta $n = 15$)
- Disco duro: 500 MB de espacio libre
- Conexión a internet (solo para descarga inicial)

Requisitos recomendados:

- Procesador: CPU de 4+ núcleos, 2.5 GHz o superior
- Memoria RAM: 16 GB para $n \leq 15$, 32 GB para $n = 20$
- Disco duro: SSD con 2 GB de espacio libre
- Conexión a internet (para descarga)

Tiempos estimados de ejecución (por caso de prueba):

Tamaño del sistema	Tiempo estimado	RAM recomendada
$n = 3$ (8 estados)	< 1 segundo	8 GB
$n = 10$ (1024 est.)	~2–4 segundos	8 GB
$n = 15$ (32768 est.)	~5–17 segundos	8 GB
$n = 20$ (1M estados)	~7 minutos	16–32 GB
$n = 22$ (4M estados)	> 3,5 horas	32+ GB
$n = 25$ (33M est.)	~67 min total*	32+ GB

* $n = 25$ requiere el benchmark rápido con KL+MC-EMD (aproximado). El tiempo mostrado es para los 50 casos completos, no por caso.

2.3. Software

Requisitos de software obligatorios:

- Python 3.11 o superior (3.9.13 mínimo)
- pip (gestor de paquetes de Python)
- Git (para clonar el repositorio, opcional)

Bibliotecas de Python (se instalan automáticamente):

- `numpy`: cálculos numéricos y álgebra lineal
- `pandas`: manejo de datos tabulares
- `openpyxl`: lectura/escritura de archivos Excel
- `pyphi`: integración con Teoría de Información Integrada
- `pyemd`: cálculo de Earth Mover's Distance
- `pyinstrument`: medición de rendimiento
- `colorama`: salida de terminal coloreada

3. Instalación paso a paso

3.1. Descarga del proyecto

Opción A: Clonar el repositorio Git (recomendado)

1. Abra una terminal (símbolo del sistema en Windows, terminal en Linux/macOS).
2. Ejecute el siguiente comando:

```
1 git clone <URL_DEL_REPOSITORIO>
```

Reemplace <URL_DEL_REPOSITORIO> con la dirección del repositorio proporcionada por su instructor o equipo.

3. Ingrese a la carpeta del proyecto:

```
1 cd Proyecto-Analisis-20261
```

Opción B: Descargar archivo comprimido

1. Acceda al repositorio del proyecto desde su navegador web.
2. Haga clic en “Code” y luego en “Download ZIP”.
3. Extraiga el contenido del ZIP en una carpeta de su preferencia.
4. Abra una terminal y navegue hasta la carpeta extraída.

3.2. Instalación de Python

Si ya tiene Python 3.11+ instalado, puede saltar al paso 3.3.

Windows:

1. Vaya a <https://www.python.org/downloads/>
2. Descargue Python 3.11 o superior (64 bits recomendado).
3. Ejecute el instalador.
4. **Importante:** Marque la casilla “Add Python to PATH” antes de hacer clic en “Install Now”.
5. Espere a que termine la instalación.
6. Verifique la instalación abriendo una terminal y escribiendo:

```
1 python --version
```

Debería ver algo como: Python 3.11.x

Linux (Ubuntu/Debian):

Ejecute en la terminal:

```
1 sudo apt update
2 sudo apt install python3 python3-pip python3-venv -y
3 python3 --version
```

macOS:

Descargue el instalador desde <https://www.python.org/downloads/> o use:

```
1 brew install python@3.11
```

3.3. Instalación del gestor de entornos uv

KGeoMIP utiliza “uv” como gestor de entornos virtuales. Es una herramienta moderna más rápida que pip o poetry.

1. En la terminal, ejecute:

```
1 pip install uv
```

2. Verifique la instalación:

```
1 uv --version
```

Si prefiere no usar uv, puede usar pip y venv directamente (ver sección 3.6 para instrucciones alternativas).

3.4. Creación del entorno virtual e instalación de dependencias

Una vez dentro de la carpeta del proyecto, ejecute:

```
1 cd GeoMIP/src/Method2_Dynamic_Programming_Reformulation
2 uv sync
```

Este comando:

1. Crea un entorno virtual en la carpeta `.venv/`
2. Lee el archivo `pyproject.toml` para identificar las dependencias
3. Descarga e instala todas las bibliotecas necesarias

El proceso puede tomar de 1 a 5 minutos dependiendo de su conexión a internet. Si todo sale bien, verá un mensaje indicando que el entorno está listo.

3.5. Verificación de instalación correcta

Para confirmar que todo funciona correctamente, ejecute la suite de pruebas:

```
1 uv run pytest ../../../../tests/ -v
```

Debería ver una salida similar a:

```

1 ===== test session starts =====
2 ...
3 tests/test_end.py ..... [ 20%]
4 tests/test_geometric_n3.py ..... [ 50%]
5 tests/test_kpart_coherence.py ..... [ 68%]
6 tests/test_solutions.py ..... [100%]
7 ===== 48 passed in XX.XXs =====

```

Si ve “48 passed”, la instalación ha sido exitosa.

También puede ejecutar una prueba de humo rápida:

```

1 uv run python ../../smoke_test.py

```

3.6. Solución de problemas comunes de instalación

Problema: “uv no se reconoce como un comando interno o externo”

Solución: Asegúrese de que pip está actualizado y reinstale uv:

```

1 python -m pip install --upgrade pip
2 pip install uv

```

Problema: Error de versión de Python

Solución: Verifique que tiene Python 3.9.13 o superior:

```

1 python --version

```

Si tiene una versión anterior, descargue la versión más reciente desde python.org.

Problema: Error al instalar dependencias (pyphi)

Solución: pyphi puede requerir compilación en algunos sistemas. En Windows, asegúrese de tener Microsoft C++ Build Tools instalado: <https://visualstudio.microsoft.com/visual-cpp-build-tools/>

Problema: Usar pip en lugar de uv (alternativa)

Solución: Si uv no funciona, puede usar pip directamente:

```

1 cd GeoMIP/src/Method2_Dynamic_Programming_Reformulation
2 python -m venv .venv
3 .venv\Scripts\activate (Windows)
4 source .venv/bin/activate (Linux/macOS)
5 pip install -e .

```

Problema: Espacio en disco insuficiente

Solución: Los archivos de TPM grandes (N20A.csv, N22A.csv) pueden ocupar varios cientos de MB. Asegúrese de tener al menos 2 GB disponibles.

4. Video tutorial

4.1. Enlace al video

El video tutorial completo de instalación y uso está disponible en:

<https://youtu.be/WQj4LdI5mIE?feature=shared>

Recomendación: vea el video antes de comenzar la instalación para tener una visión general del proceso.

5. Guía de uso básico

5.1. Preparación de datos de entrada

5.1.1. Archivos de Matriz de Probabilidad de Transición (TPM)

Las TPMs son archivos CSV (valores separados por comas) que describen la dinámica del sistema. Cada fila representa un estado posible del sistema en el instante $t-1$ (pasado), y cada columna representa la probabilidad de que un nodo específico esté en estado 1 en el instante t (futuro).

Formato del archivo CSV:

- Primera fila: encabezados con nombres de nodos (ej: A, B, C, ...)
- Filas siguientes: valores binarios (0 o 1) o probabilísticos (0.0 a 1.0)
- Total de filas: 2^n (donde n es el número de nodos)
- Total de columnas: n (una por nodo)

Ejemplo para un sistema de 3 nodos (N3C.csv):

```
1 A,B,C
2 0,0,0
3 0,0,1
4 0,1,0
5 0,1,1
6 1,0,0
7 1,0,1
8 1,1,0
9 1,1,1
```

Los archivos TPM predefinidos se encuentran en:

```
1 GeoMIP/data/samples/
```

El proyecto incluye TPMs para sistemas de 3 a 22 nodos:

- N3A.csv, N3B.csv, N3C.csv (3 nodos — 8 filas)
- N4A.csv, N4B.csv, N4C.csv (4 nodos — 16 filas)
- N5A.csv, N5B.csv (5 nodos — 32 filas)
- N10A.csv (10 nodos — 1024 filas)
- N15A.csv, N15B.csv (15 nodos — 32768 filas)
- N20A.csv (20 nodos — 1048576 filas)

5.1.2. Archivo de configuración Excel

El archivo `Pruebas_Metodo2.xlsx` (en `GeoMIP/results/`) contiene las configuraciones de los casos a ejecutar. Cada fila del Excel especifica:

- Archivo TPM a utilizar
- Estado inicial del sistema
- Alcance (nodos futuros a considerar)
- Mecanismo (nodos presentes a considerar)
- Condición (nodos de fondo fijos)

Ejemplo de fila en el Excel:

- TPM: `N10A.csv`
- Estado inicial: `1111111111`
- Alcance: `ABCDEFGHJIJ`
- Mecanismo: `ABCDEFGHJIJ`
- Condición: (vacío)

5.1.3. Estructura de directorios

Asegúrese de que la estructura de carpetas sea la siguiente:

```
1 Proyecto-Analisis-20261/  
2   |-- GeoMIP/  
3   |   |-- data/  
4   |   |   |-- samples/           (aqui van los archivos CSV)  
5   |   |-- results/              (aqui se generan los resultados)  
6   |   |-- src/  
7   |       |-- Method2_Dynamic_Programming_Reformulation/  
8   |           |-- src/           (codigo fuente)  
9   |-- tests/                    (pruebas)  
10  |-- docs/                     (documentacion)
```

5.2. Ejecución básica

5.2.1. Ejecución desde Excel (GeometricSIA + KPartition)

Este modo ejecuta automáticamente todos los casos definidos en el archivo `Pruebas_Metodo2.xlsx`.

1. Abra una terminal en la carpeta del proyecto.
2. Navegue hasta el directorio de ejecución:

```
1 cd GeoMIP/src/Method2_Dynamic_Programming_Reformulation
```

3. Ejecute el programa principal:

```
1 uv run exec.py
```

4. Espere a que el programa termine. La salida se mostrará en la terminal y los resultados se guardarán en:

```
1 GeoMIP/results/resultados_Geometric.xlsx
```

5.2.2. Ejecución de un caso específico (debug)

Para probar un caso concreto sin necesidad de configurar el Excel, use el script de depuración:

1. Navegue al directorio de ejecución (igual que arriba).

2. Ejecute:

```
1 uv run python debug_kpart.py
```

Por defecto, este script analiza el sistema N3 con el estado “100” y $k = 3$. Puede modificar los parámetros directamente en el archivo `debug_kpart.py` para probar otras configuraciones.

5.2.3. Ejecución del benchmark completo

El benchmark ejecuta múltiples estrategias sobre conjuntos predefinidos de casos y genera un reporte completo en Excel.

Para sistemas pequeños ($n = 10$ y $n = 15$, rápido):

```
1 uv run python ../../benchmark.py --n 10 15 --timeout 300
```

Para sistemas grandes ($n = 20$, puede tomar horas):

```
1 uv run python ../../benchmark.py --n 20 --timeout 21600
```

Para benchmark aproximado KL+MC-EMD ($n = 10$ a $n = 25$, más rápido):

```
1 uv run python ../../benchmark_aprox.py --n 10 15 20 22 25
```

Para benchmark rápido MCTS/KL+MC-EMD ($n = 10$ a $n = 25$, el más rápido):

```
1 uv run python ../../benchmark_rapido.py --n 10 15 20 22 25
```

Explicación de parámetros:

- `-n 10 15`: analiza sistemas de 10 y 15 nodos
- `-timeout 300`: tiempo máximo por estrategia (300 segundos)

Los resultados se guardan en:

- Exacto: `GeoMIP/data/results/n{n}/benchmark_YYYY-MM-DD_HHhMM.xlsx`
- Aprox: `GeoMIP/data/results/aprox/n{n}/n{n}_aprox*.xlsx`
- Rápido: `GeoMIP/data/results/rapido/n{n}/n{n}_rapido*.xlsx`

5.3. Interpretación de resultados

5.3.1. Salida en consola

Cuando ejecuta un análisis, la salida típica en consola es:

```
1 =====
2 Resultado - Estrategia: GeometricSIA
3 Perdida (phi): 0.076590
4 Particion: {A,B,C,D,E} | {F,G,H,I,J}
5 Tiempo: 1.58 s
6 k: 2
7 =====
```

Interpretación:

- “Estrategia”: El algoritmo utilizado.
- “Pérdida (phi)”: Valor numérico de la pérdida de información. Valores cercanos a 0 indican buena partición.
- “Partición”: Cómo se agrupan los nodos. “|” separa las partes.
- “Tiempo”: Cuánto tardó el cálculo.

Si una estrategia no converge a tiempo, verá:

```
1 Geo=T    (GeometricSIA Timeout)
2 KP3=T    (KPartition k=3 Timeout)
```

5.3.2. Archivos de resultados Excel

Los archivos Excel generados contienen una fila por caso analizado con las siguientes columnas:

- **tpm**: nombre del archivo TPM usado
- **n**: número de nodos del sistema
- **purview_texto**: alcance (nodos futuros)
- **mecanismo_texto**: mecanismo (nodos presentes)
- **k**: número de partes
- **estrategia**: nombre del algoritmo
- **particion**: descripción de la partición encontrada
- **perdida_emd**: valor de ϕ (pérdida de información)
- **tiempo_ms**: tiempo de ejecución en milisegundos
- **memoria_mb**: memoria utilizada en megabytes
- **convergio**: “1” si convergió, “0” si timeout

Puede abrir estos archivos con Microsoft Excel, LibreOffice Calc o Google Sheets para analizar los resultados.

5.3.3. Archivos de log

Los archivos de log (en `GeoMIP/results/logs/`) contienen el detalle completo de la ejecución, incluyendo:

- Fecha y hora de cada operación
- Parámetros de cada caso
- Resultados intermedios
- Mensajes de error si los hay

Para sistemas grandes ($n \geq 20$), los logs se guardan automáticamente en:

```
1 GeoMIP/results/logs/n20_plus/
```

5.4. Ejecución de la suite de pruebas

Para verificar que todo funciona correctamente después de la instalación o realizar cambios:

```
1 cd GeoMIP/src/Method2_Dynamic_Programming_Reformulation
2 uv run pytest ../../../../tests/ -v
```

Esto ejecuta 48 pruebas que verifican:

- Cálculo correcto de EMD (7 pruebas)
- Reproducción de resultados de referencia N3 (10 pruebas)
- Coherencia de k -particiones (6 pruebas)
- Campos mínimos en resultados (11 pruebas)
- Heurística KL+MC-EMD (7 pruebas)
- Partición aleatoria (4 pruebas)
- MC-EMD (3 pruebas)

6. Opciones y parámetros avanzados

6.1. Parámetros del benchmark

El script `benchmark.py` acepta los siguientes parámetros:

- n, -n** Lista de tamaños de red a analizar Ejemplo: `-n 10 15 20`
Valores disponibles: 6, 10, 15, 20, 22, 25
- timeout** Tiempo máximo (segundos) por estrategia por caso Predeterminado: 600
Recomendado: 300 para $n \leq 15$, 21600 para $n \geq 20$
- k, -k** Valores de k a analizar Ejemplo: `-k 3 4 5`
Predeterminado: 3, 4, 5
- no-run-log** Desactiva la generación automática de archivos de log (útil si solo interesa el Excel de resultados)

Ejemplo completo:

```
1 uv run python ../../benchmark.py --n 10 15 --k 3 4 --timeout 300
```

Esto analiza sistemas de 10 y 15 nodos con $k = 3$ y $k = 4$, con timeout de 300 segundos por estrategia.

6.2. Estrategias disponibles

El software incluye las siguientes estrategias de análisis:

- GeometricSIA ($k = 2$):** ■ Programación dinámica sobre el hipercubo de estados
- **Exacta** para bipartición (encuentra la mejor partición posible)
 - Más rápida que QNodes (hasta $4\times$ más rápida)
 - Recomendada **siempre** para $k = 2$
- QNodes ($k = 2$):** ■ Heurística greedy de nodos
- **Aproximada** (no garantiza optimalidad)
 - Más lenta que GeometricSIA
 - Mantenida solo por compatibilidad con versiones anteriores
- KPartitionSIA ($k = 3, 4, 5$):** ■ Extensión a múltiples partes
- Incluye varias heurísticas internas (ver sección [6.3](#))
 - Usa GeometricSIA como base para $k = 2$
- KL+MC-EMD ($k = 2, 5, n \geq 25$):** ■ Algoritmo de aproximación puro: omite el paso de partición exacta
- Fase 1: Partición aleatoria del sistema
 - Fase 2: Refinamiento con Kernighan-Lin usando estimación MC-EMD
 - Única estrategia viable para $n = 25$
 - Resultados inherentemente aproximados (sin referencia exacta)

6.3. Heurísticas de KPartitionSIA

El algoritmo KPartitionSIA incluye cinco heurísticas para encontrar k -particiones:

GREEDY (Voraz): ■ Estrategia: extrae nodos uno a uno de las partes existentes

- Ventaja: rápido, buena solución base
- Desventaja: puede quedarse en mínimos locales
- Uso recomendado: cuando la velocidad es prioritaria

KL (Kernighan-Lin): ■ Estrategia: intercambia nodos entre partes existentes

- Ventaja: **mejor calidad**, nunca peor que Greedy
- Desventaja: ligeramente más lento ($1.1\times$ a $2.3\times$)
- Uso recomendado: **siempre** que se requiera calidad

CLUSTERING (Agrupamiento jerárquico): ■ Estrategia: agrupa nodos por similitud de perfiles

- Ventaja: rápido ($O(n^3)$)
- Desventaja: produce pérdidas 9–10 $\times$ peores que Greedy
- Uso: solo como comparación académica

SPECTRAL (Espectral): ■ Estrategia: corte espectral normalizado

- Ventaja: rápido $O(n^3)$
- Desventaja: **no adecuado** para este problema
- Uso: solo como baseline negativo

MCTS + MC-EMD: ■ Estrategia: búsqueda por árbol Monte Carlo + estimación EMD

- Ventaja: única viable para $n \geq 20$ ($n \leq 22$)
- Desventaja: resultados aproximados (error $\sim 2\%$)
- Uso: automático para $20 \leq n \leq 22$

KL + MC-EMD: ■ Estrategia: partición aleatoria + refinamiento Kernighan-Lin con estimación Monte Carlo de la EMD

- Ventaja: única viable para $n = 25$; completa los 50 casos en ~ 67 minutos total
- Desventaja: resultados aproximados, sin referencia exacta para validación cruzada
- Uso: automático para $n \geq 25$; también disponible via `forzar_heuristica="kl_mc"`

6.4. Parámetro de timeout

El timeout controla cuánto tiempo máximo puede ejecutarse cada estrategia antes de ser cancelada. Es importante configurarlo adecuadamente:

- $n \leq 10$: timeout = 120 s (suficiente para todas las estrategias)

- $n \leq 15$: timeout = 300 s (tiempo recomendado)
- $n = 20$: timeout = 21600 s (6 horas por estrategia)
- $n = 22$: timeout = 43200 s (12 horas, recomendado)
- $n = 25$: timeout = 600 s (10 min por estrategia con KL+MC-EMD)

Cada caso de prueba ejecuta varias estrategias:

- Para $n \leq 15$: 8 estrategias (QNodes $k = 2$, Geo $k = 2$, Greedy/KL $\times k = 3, 4, 5$)
- Para $20 \leq n \leq 22$: 7 estrategias (sin QNodes)
- Para $n = 25$: 1 estrategia (KL+MC-EMD, la única viable)

Por lo tanto, el tiempo total por caso es aproximadamente:

$$\text{Tiempo_total} = \text{timeout} \times \text{numero_estrategias}$$

Ejemplo: para $n = 20$ con timeout=21600:

$$\text{Tiempo_maximo_por_caso} = 21600 \times 7 = 151200 \text{ s} = 42 \text{ horas}$$

Sin embargo, en la práctica las estrategias convergen mucho antes del timeout (media de 7.3 min por caso para $n = 20$).

6.5. Optimización de rendimiento

Consejos para mejorar el rendimiento:

1. **Reducir el subsistema:** Si su sistema tiene más de 15 nodos, considere analizar solo un subconjunto de nodos (alcance y mecanismo parciales). Esto reduce drásticamente el tiempo.
2. **Usar GeometricSIA para $k = 2$:** Es $2.8\text{--}4.1\times$ más rápido que QNodes con la misma calidad de resultado.
3. **Evitar QNodes para $n \geq 15$:** QNodes escala peor que GeometricSIA. Para $n = 15$, QNodes toma ~ 17 s mientras GeometricSIA toma ~ 4 s.
4. **Para $k \geq 3$, usar KL:** Aunque es $1.1\text{--}2.3\times$ más lento que Greedy, la mejora en calidad (31–79 %) compensa ampliamente el tiempo adicional.
5. **Desactivar el profiler:** Si no necesita mediciones de rendimiento detalladas, el profiler PyInstrument añade overhead. El benchmark lo desactiva por defecto.
6. **Memoria:** Para $n \geq 20$, asegúrese de tener suficiente RAM (16–32 GB) y cierre otras aplicaciones durante la ejecución.
7. Para $n \geq 21$, la TPM se carga en formato `float32` desde archivos `.npy` binarios mediante carga diferida (streaming). Esto reduce el uso de RAM de ~ 34 GB a ~ 6.8 GB para $n = 25$.

8. Para $n = 25$, use exclusivamente `benchmark_rapido.py` o `benchmark_aprox.py` con la heurística KL+MC-EMD. No intente `benchmark.py` con $n = 25$ (requiere recursos desproporcionados).
9. **MC-EMD (Monte Carlo EMD)**: activo automáticamente para $n \geq 21$. Estima la EMD mediante muestreo de columnas en lugar de calcularla exactamente, reduciendo el costo de $O(2^n)$ a $O(m)$.

6.6. Cache de resultados

El software implementa un sistema de cache automático que evita recalcular el mismo caso múltiples veces:

Cache de subsistemas: Cuando varias estrategias procesan el mismo caso (mismos parámetros de entrada), el subsistema se construye una sola vez y se reutiliza. Esto reduce hasta $8\times$ el tiempo de preparación por caso.

Cache de `find_mip`: GeometricSIA memoiza los resultados de su búsqueda BFS. Cuando KPartitionSIA necesita la bipartición base, la obtiene del cache en lugar de recalcularla.

El cache se limpia automáticamente entre casos diferentes. No requiere configuración por parte del usuario.

7. Solución de problemas

7.1. Errores comunes

Error: “ModuleNotFoundError: No module named 'src'”

Causa: El comando se ejecutó desde el directorio incorrecto.

Solución: Navegue a `GeoMIP/src/Method2_Dynamic_Programming_Reformulation` antes de ejecutar.

Error: “FileNotFoundError: No such file or directory: '...N3C.csv’

Causa: Los archivos TPM no están en la ubicación esperada.

Solución: Verifique que los archivos CSV estén en `GeoMIP/data/samples/`.

Error: “ValueError: k (7) debe ser $\leq$ numero de nodos totales”

Causa: Se solicitó una partición con más partes que nodos disponibles.

Solución: Use $k \leq$ número de nodos del subsistema.

Error: “ERROR_INCOMPATIBLE_SIZES”

Causa: Los tamaños del estado inicial, condición, alcance y mecanismo no coinciden.

Solución: Verifique que todos los parámetros tengan la misma longitud (igual al número de nodos de la TPM).

Error: Tiempo de ejecución excesivo (>30 min)

Causa: El subsistema tiene más de 14–15 nodos activos.

Solución: Use timeout adecuado (`-timeout 21600`) o reduzca el tamaño del subsistema.

7.2. Problemas de instalación

Problema: “pip no se reconoce como un comando”

Solución: En Windows, asegúrese de haber marcado “Add Python to PATH” durante la instalación. Si no, reinstale Python marcando esa opción.

Problema: “uv sync” falla con error de compilación

Solución: Algunas dependencias (como `pyphi`) requieren compilación C. En Windows, instale Microsoft C++ Build Tools desde: <https://visualstudio.microsoft.com/visual-cpp-build-tools/>

Problema: Error “python no encontrado” en Linux

Solución: Use `python3` en lugar de `python`:

```
1 python3 --version
```

Problema: Conflictos con otros proyectos de Python

Solución: Use siempre el entorno virtual de uv. El comando `uv sync` crea un entorno aislado que no interfiere con otros proyectos.

7.3. Problemas de ejecución

Problema: El programa no muestra nada en la terminal

Solución: Espere unos segundos (los sistemas grandes pueden tomar tiempo en cargar). Si tras 30 segundos no hay salida, verifique que la TPM existe y tiene el formato correcto.

Problema: El programa se cuelga (no responde)

Solución: Para $n \geq 20$, el cálculo puede tomar horas. Use la opción `-timeout` para limitar el tiempo de espera. Si realmente está colgado, cierre la terminal y reduzca el tamaño del sistema.

Problema: Consume demasiada memoria

Solución: Para $n = 20$, el software puede usar 16+ GB de RAM. Cierre otras aplicaciones. Si el problema persiste, analice subsistemas más pequeños (alcance/-mecanismo parcial).

Problema: Resultados inesperados (pérdida muy alta)

Solución: Verifique que la TPM tenga el formato correcto. Si la TPM es probabilística (valores entre 0 y 1), asegúrese de que las filas sumen 1 (o valores coherentes).

7.4. Problemas con datos de entrada

Problema: El archivo CSV no se lee correctamente

Solución: Verifique que el CSV no tenga filas en blanco, que los valores sean 0 o 1 (para TPMs binarias), y el separador sea coma.

Problema: Error en el alcance o mecanismo

Solución: Los nombres de nodos en alcance y mecanismo deben coincidir con las columnas de la TPM. Use letras mayúsculas (A, B, C, ...).

Problema: El número de filas de la TPM no es 2^n

Solución: Para un sistema de n nodos, la TPM debe tener exactamente 2^n filas. Si tiene menos, el sistema está incompleto.

7.5. Preguntas frecuentes (FAQ)

P: ¿Cuál es la diferencia entre GeometricSIA y QNodes?

R: GeometricSIA es exacto para $k = 2$ y más rápido. QNodes es heurístico (aproximado). Siempre prefiera GeometricSIA para $k = 2$.

P: ¿Qué valor de k debo usar?

R: Depende del sistema:

- $n \leq 10$: $k = 3$ suele ser óptimo (82 % de los casos)
- $n = 11\text{--}15$: $k = 2$ es suficiente en el 62 % de los casos
- Si no está seguro, ejecute con $k = 2, 3, 4, 5$ y compare los resultados.

P: ¿Por qué algunos resultados muestran “Inviabile”?

R: Ocurre cuando k es mayor que el número total de nodos del subsistema. Por ejemplo, no se puede hacer una 7-partición de un sistema de 6 nodos.

P: ¿Puedo analizar mi propio sistema?

R: Sí. Solo necesita crear un archivo CSV con la TPM de su sistema y agregar una fila en `Pruebas_Metodo2.xlsx` con los parámetros deseados.

P: ¿Los resultados son reproducibles?

R: Sí. El software utiliza una semilla fija para los componentes aleatorios (cuando aplica), por lo que los resultados son determinísticos.

P: ¿Qué hacer si el benchmark tarda demasiado?

R: Use `-timeout` para limitar cada estrategia. O ejecute solo los tamaños de red que necesita (ej: `-n 10` en lugar de `-n 10 15 20`).

P: ¿Puedo analizar un sistema de 25 nodos?

R: Sí, usando el benchmark rápido (`benchmark_rapido.py`) o el benchmark aproximado (`benchmark_aprox.py`). El algoritmo KL+MC-EMD completa los 50 casos en ~ 67 minutos con 32 GB de RAM.

P: ¿Por qué no hay resultados exactos para $n = 25$?

R: Para $n = 25$, la TPM tiene 33 millones de filas (~ 3.4 GB). El algoritmo exacto (GeometricSIA) requeriría $O(2^n)$ operaciones por evaluación, lo que es computacionalmente inviable. KL+MC-EMD omite el paso exacto y usa partición aleatoria + refinamiento.

P: ¿Cuál es la diferencia entre `benchmark.py`, `benchmark_rapido.py` y `benchmark_aprox.py`?

R: `benchmark.py` ejecuta estrategias exactas y heurísticas estándar (recomendado para $n \leq 22$). `benchmark_rapido.py` usa MCTS+MC-EMD para $n \leq 22$ y KL+MC-EMD para $n \geq 25$ (el más versátil). `benchmark_aprox.py` usa exclusivamente KL+MC-EMD para todos los tamaños.

P: ¿Qué significa “KL+MC-EMD”?

R: Es un algoritmo de dos fases: (1) Partición aleatoria del sistema, (2) Refinamiento iterativo mediante Kernighan-Lin que usa estimación Monte Carlo de la EMD en lugar del cálculo exacto. Es la única estrategia viable para sistemas de 25 nodos.

8. Ejemplos y tutoriales

8.1. Tutorial básico: Sistema de 3 nodos (N3C)

Este tutorial le guiará a través del análisis completo de un sistema pequeño de 3 nodos.

Paso 1: Verificar la instalación

Abra una terminal y ejecute:

```
1 cd GeoMIP/src/Method2_Dynamic_Programming_Reformulation
2 uv run python -c "print('Instalacion OK')"
```

Debería ver: Instalacion OK

Paso 2: Ejecutar la suite de pruebas

Ejecute:

```
1 uv run pytest ../../../../tests/ -v --tb=short
```

Debería ver 48 pruebas pasadas. Si alguna falla, revise la instalación.

Paso 3: Ejecutar un caso de prueba con $k = 2$

```
1 uv run python debug_kpart.py
```

Este script analiza el sistema N3 con estado “100” y encuentra la mejor bipartición ($k = 2$).

La salida debería ser similar a:

```
1 Particion: {A,B,C} | Perdida: X.XXXXXX
2 Estrategia: ... | Tiempo: X.XXs
```

Paso 4: Interpretar el resultado

- La partición muestra cómo se agrupan los 3 nodos.
- El valor de pérdida (ϕ) indica cuánta información se pierde.
- Un valor cercano a 0 indica una partición que preserva casi toda la información del sistema.

Paso 5: Probar con $k = 3$

Modifique el archivo `debug_kpart.py` cambiando el valor de k de 3 a 2 (o viceversa según la configuración actual) y ejecute nuevamente. Compare los valores de pérdida: $k = 3$ debería tener pérdida mayor o igual que $k = 2$ (porque dividir en más partes siempre pierde más información).

8.2. Caso de estudio intermedio: Sistema de 10 nodos (N10A)

Este tutorial analiza un sistema de 10 nodos con diferentes valores de k .

Paso 1: Ejecutar benchmark para $n = 10$

```
1 uv run python ../../benchmark.py --n 10 --timeout 120
```

Esto ejecutará 49 casos de prueba con todas las estrategias. Tiempo estimado: ~3 minutos.

Paso 2: Revisar los resultados

Al finalizar, abra el archivo generado en:

```
1 GeoMIP/results/benchmark_YYYY-MM-DD_HHhMM.xlsx
```

Observe las columnas:

- `perdida_emd_Geo_k2`: pérdida de GeometricSIA para $k = 2$
- `perdida_emd_KL_k3`: pérdida de Kernighan-Lin para $k = 3$
- Compare los valores y observe cómo `KL_k3` produce menor pérdida que `Greedy_k3` (columna `perdida_emd_QN_k3`).

Paso 3: Comparar resultados

Para el caso típico ABCDEFGHIJ/ABCDEFGHIJ:

- GeometricSIA $k = 2$: pérdida ~ 0.077
- KL $k = 3$: pérdida ~ 0.034 (mejor que $k = 2$ en algunos casos)
- Greedy $k = 3$: pérdida ~ 0.165 (peor que KL)

Concluya que KL encuentra mejores particiones que Greedy.

8.3. Ejemplo avanzado: Benchmark con $n = 20$

Advertencia: Este ejemplo puede tomar varias horas. Ejecute solo si tiene suficiente tiempo y recursos.

Paso 1: Preparación

Asegúrese de tener al menos 16 GB de RAM disponibles y tiempo suficiente (~ 6 horas para los 50 casos).

Paso 2: Ejecutar benchmark

```
1 uv run python ../../benchmark.py --n 20 --timeout 21600
```

Paso 3: Monitorear el progreso

Revise el archivo de log en:

```
1 GeoMIP/results/logs/n20_plus/
```

Allí encontrará el detalle de cada caso a medida que se completa.

Paso 4: Interpretar resultados

Tras la ejecución, abra:

```
1 GeoMIP/results/n20/n20_completo_YYYY-MM-DD_HHhMM.xlsx
```

Observe:

- Tiempo medio de GeometricSIA: ~ 3.9 min por caso
- Tiempo total medio por caso: ~ 7.3 min (todas las estrategias)
- Las columnas `KL_k3` tienen menor pérdida que `QN_k3`
- La regla heurística integrada recomienda $k = 3$ para todos los casos

8.4. Ejemplo $n = 25$: Benchmark aproximado con KL+MC-EMD

Advertencia: Los resultados son aproximados. No existe referencia exacta para validación cruzada.

Paso 1: Asegúrese de tener 32 GB de RAM y TPMs en formato `.npy` binario en `GeoMIP/data/samples/` (formato `float32`, carga streaming).

Paso 2: Ejecutar benchmark aproximado:

```
1 cd GeoMIP/src/Method2_Dynamic_Programming_Reformulation
2 uv run python ../../benchmark_aprox.py --n 25
```

Tiempo estimado total (50 casos): ~67 minutos.

Paso 3: Resultados

Los resultados se guardan en:

```
1 GeoMIP/data/results/aprox/n25/n25_aprox_YYYY-MM-DD_HHhMM.xlsx
```

Columnas relevantes:

- `k`: número de partes
- `perdida_emd`: valor de ϕ estimado por MC-EMD
- `tiempo_ms`: tiempo por caso
- `convergio`: “1” si completó, “0” si timeout

Paso 4: Interpretación

Para $n = 25$, la heurística recomienda automáticamente el mejor k . Los valores de pérdida son comparables entre sí dentro del mismo experimento, pero **no** deben compararse directamente con resultados exactos de $n \leq 20$ debido a la estimación Monte Carlo de la EMD.

9. Referencia rápida

9.1. Comandos principales

Descripción	Comando
Ejecutar desde Excel	<code>uv run exec.py</code>
Benchmark exacto ($n \leq 22$)	<code>uv run python ../../benchmark.py -n <tam</code>
Benchmark aprox. KL+MC-EMD ($n \leq 25$)	<code>uv run python ../../benchmark_aprox.py -</code>
Benchmark rápido MCTS/KL+MC-EMD ($n \leq 25$)	<code>uv run python ../../benchmark_rapido.py</code>
Generar gráfica comparativa	<code>uv run python ../../gen_comparativa.py</code>
Ejecutar pruebas	<code>uv run pytest ../../tests/ -v</code>
Validar TPMs	<code>uv run python ../../data/validate_tpms.p</code>
Depuración de caso específico	<code>uv run python debug_kpart.py</code>
Ejecutar smoke test	<code>uv run python ../../smoke_test.py</code>
Ejecutar k -partition batch	<code>uv run python run_kpart.py</code>

9.2. Tabla de parámetros

Parámetro	Valores	Descripción
<code>-n</code>	6,10,15,20,22,25	Tamaños de red
<code>-timeout</code>	1–86400 s	Timeout por estrategia
<code>-k, -k</code>	2,3,4,5	Valores de k
<code>-no-run-log</code>	flag	Desactiva log automático
<code>forzar_heurist.</code>	greedy/KL/kl_mc/mcts/mcts_mc	Heurística específica
<code>-heurísticas</code>	kl_mc,mcts_mc,mcts	Heurísticas para <code>benchmark_rapido.py</code>

9.3. Formato de archivos

Archivos de entrada (CSV — TPMs):

- Formato: CSV con encabezados
- Columnas: n columnas (una por nodo, letras A, B, C, ...)
- Filas: 2^n filas (cada fila = un estado posible)
- Valores: 0 o 1 (binaria) o float 0.0–1.0 (probabilística)
- Ubicación: `GeoMIP/data/samples/`

Archivos de configuración (Excel):

- Formato: `.xlsx`
- Hojas: configuraciones de casos
- Columnas: TPM, estado_inicial, purview, mecanismo, condición
- Ubicación: `GeoMIP/results/Pruebas_Metodo2.xlsx`

Archivos de salida (Excel):

- Formato: `.xlsx` con múltiples hojas
- Hojas: resultados por n , resumen heurístico
- Columnas: por estrategia/ k con pérdida, tiempo, memoria
- Ubicación: `GeoMIP/results/`

9.4. Glosario

EMD (Earth Mover's Distance): Medida matemática de distancia entre dos distribuciones de probabilidad. En términos simples: “el esfuerzo mínimo para convertir una distribución en otra”.

IIT (Integrated Information Theory): Teoría científica que busca medir la cantidad de información integrada generada por un sistema. Relacionada con el estudio de la conciencia.

k (número de partes): Cantidad de grupos en que se divide el sistema. Ejemplo: $k = 3$ significa dividir en 3 partes.

MIP (Minimum Information Partition): La partición del sistema que causa la menor pérdida de información posible. Es la “mejor” forma de dividir el sistema.

ϕ (pérdida de información): Valor numérico que indica cuánta información se pierde al dividir el sistema. $\phi = 0$ significa pérdida nula (partes independientes).

TPM (Transition Probability Matrix): Tabla que describe la dinámica del sistema: dado un estado actual, cuál es la probabilidad de cada estado futuro.

Heurística: Método de búsqueda que encuentra una buena solución sin garantizar que sea la mejor posible. Útil cuando el problema es demasiado grande para buscar todas las opciones.

Greedy (Algoritmo voraz): Estrategia que toma la mejor decisión local en cada paso, esperando que esto lleve a una buena solución global.

Kernighan-Lin (KL): Algoritmo de refinamiento de particiones que intercambia elementos entre grupos para mejorar la calidad de la solución.

Cache: Almacenamiento temporal de resultados para evitar recalcular operaciones que ya se realizaron.

Timeout: Límite de tiempo máximo permitido para una operación. Si se excede, la operación se cancela.

Benchmark: Conjunto de pruebas estandarizado para medir y comparar el rendimiento de diferentes algoritmos.

MC-EMD (Monte Carlo EMD): Estimación aproximada de la EMD mediante muestreo aleatorio de columnas. Útil para sistemas grandes ($n \geq 21$) donde el cálculo exacto es inviable. Reduce el costo de $O(2^n)$ a $O(m)$ con m muestras.

KL+MC-EMD (Kernighan-Lin con Monte Carlo EMD): Heurística de dos fases para $n \geq 25$: (1) partición aleatoria inicial, (2) refinamiento iterativo con Kernighan-Lin usando estimación MC-EMD. Omite el paso exacto de partición.

Partición Aleatoria: Estrategia que asigna nodos a partes al azar como punto de partida. Usada en KL+MC-EMD para evitar el costo del paso exacto (GeometricSIA) en sistemas de 25 nodos.

NCube: Estructura de datos interna que representa una distribución de probabilidad sobre el espacio de estados binarios del sistema.

Notas finales

Este manual fue desarrollado como parte del proyecto K-QGMIP de la Universidad de Caldas, Facultad de Ingeniería, curso Análisis y Diseño de Algoritmos 2026-1.

Para reportar problemas, sugerencias o solicitar soporte adicional, contacte a su instructor o al equipo de desarrollo del proyecto.

Documentación adicional disponible en:

- Manual Técnico: `docs/Manual_Tecnico_GeoMIP_EMD.txt`
- Bitácoras de desarrollo: `bitacoras/`
- README principal: `README.md`

— FIN DEL MANUAL DE USUARIO —